Tools walkthrough: one mock pass on Swift A, foam 400 mm

This sheet walks through the tools in the order of a design-and-test pass: look up a number in AVL, turn it into a CG and a nose weight, simulate the drop, make the parts, fill in a test-day row, read the video, feed the result back. Each step says what to **type** or **click** and what you should **see**. Every command here was run before it was written down; the screen text is the real output.

> **Safety rule: work in a practice copy.** The project scripts find their folders from where the script file sits. Run from a copy, they read and write only inside the copy, so practice never overwrites the project's results. Do the steps below in **PowerShell**.

Step 0: make the practice folder (once)

#	Do this	You should see
1	Press the Windows key, type `PowerShell`, press `Enter`.	A blue or black window with a prompt like `PS C:\Users\Kai>`.
2	Type and press Enter: `cd "C:\Users\Kai\OneDrive - purdue.edu\Purdue\AOL\MAV-2026\Codes"`	The prompt now ends in `...\Codes>`.
3	Type and press Enter: `py -B rigid_glider\make_practice_folder.py C:\Users\Kai\Documents\mmav_practice`	A list of what was copied (below). Takes a few seconds.
4	Type and press Enter: `cd C:\Users\Kai\Documents\mmav_practice`	The prompt ends in `...\mmav_practice>`. **Stay in this folder for every step.**

```
practice folder: C:\Users\Kai\Documents\mmav_practice
  rigid_glider: 85 scripts
  tools: 4 files (AVL, drop tracker)
  research/plate_data: 3 files (read by the pull-out simulation)
  flat_test_v2 kit tables: 176 files (read by the pull-out simulation)
  pullout/cases.csv (read by the v4 build page)
  print_these: the template and drop-sheet PDFs
  practice_inputs/SwiftA_foam400_bend6.avl
done. Run everything from this folder; it is a copy, so outputs stay here.
```

The copy holds the scripts (`rigid_glider`), AVL and the drop tracker (`tools`), the plate data and kit tables the pull-out simulation reads, the PDFs to print (`print_these`) and the AVL file for Swift A (`practice_inputs`). Delete the folder any time; nothing else depends on it.

The eight steps

1 Map: where is everything. 2 AVL: neutral point and trim. 3 Balance: CG and nose weight. 4 Pull-out: does a nose-down drop recover, how deep. 5 Parts: print and cut. 6 Test day: fill in the sheet, read the video, feed the result back. 7 CFD: what it is for. 8 Cheat sheet: every command.

Step 1: where things are

Only the folders and tools used in this walkthrough. Paths are inside `Codes\`.

Path	What it is	Question it answers
`tools\avl\avl352.exe`	AVL 3.52, the vortex-lattice program (inviscid)	Neutral point, lift, pitching moment, yaw and roll stiffness of an outline
`rigid_glider\bird_outline_v4_swift.py`	Builds the Swift outlines A, B, C: AVL runs, CG, nose weight, templates, cut files	What does Swift A look like, where does it balance, what do I cut?
`rigid_glider\walk_balance.py`	New small script: the balance arithmetic printed line by line	How does the neutral point become the CG and the nose weight?
`rigid_glider\pullout_sim.py` and `walk_pullout_one.py`	Nose-down drop simulation (flat kit wings); the second runs one drop	Does it pull out of the dive, and how deep is the first swoop?
`rigid_glider\fin_kit_v4.py`	Sizes tip fins and a tail fin with AVL, makes fin templates	What fins if it wanders in yaw?
`rigid_glider\test_plan.py`	Makes `drop_test_sheet.pdf`	What do I check and write down on test day?
`tools\drop_tracker\drop_tracker.html`	Offline page: click the glider in a phone video	How deep was the swoop, how far did it glide?
`cfd\` (runs in WSL)	OpenFOAM checks of AVL against a tunnel plate	Can AVL's moment be trusted? Not used on our wings.

Folders

```
Codes\
  tools\avl\, tools\drop_tracker\        the programs (AVL, the video tracker)
  rigid_glider\                      all the Python scripts for the glider work
  results\runs\rigid_glider\         what each script wrote, one folder per study:
      bird_outline_v4_swift\   index.html, A_templates.pdf, outlines.csv  (Swift A, B, C)
      pullout\                 index.html, cases.csv                      (drop simulation)
      fin_kit_v4\              index.html, fin_templates.pdf               (fins)
      test_plan\               drop_test_sheet.pdf                          (test day)
  research\                          notes and data (plate data is read-only input)
  cfd\                               CFD notes and scripts (cases live in WSL)
```

> **Never touch:** `data\polars`, `results\legacy`, `archive`. Never run a `rigid_glider` script from the project folder to practice: it overwrites that study's results. Practice in the copy.

To look at a finished study: in File Explorer open `Codes\results\runs\rigid_glider\bird_outline_v4_swift\` and double-click `index.html`. It opens in your browser.

Step 2: AVL by hand on Swift A

AVL is a program you talk to with short commands. Swift A at 400 mm, tail bent 6 degrees, is already written as an AVL file: `practice_inputs\SwiftA_foam400_bend6.avl` (made from the v4 geometry; its moment reference is the 37.0 mm CG). Type each line, then press `Enter`. A blank row means press Enter on its own. AVL does not care about capital letters.

#	Do this	You should see
1	`.\tools\avl\avl352.exe`	The AVL banner, a menu, and the prompt `AVL c>`.
2	`plop` then `g` then a blank line	Two small menus; the line `Graphics-enable flag F` means the plot window is now off (so AVL cannot hang on it).
3	`load practice_inputs\SwiftA_foam400_bend6.avl`	`Configuration: SwiftA_foam400_bend6` and the surfaces being built (Wing, Tail).
4	`oper`	The operating-point menu, prompt `.OPER (case 1/1) c>`.
5	`a c 0.3`	Sets 'alpha is whatever gives CL = 0.3'. The table shows `A lpha -> CL = 0.3000`.
6	`x`	A few iteration lines, then **Total Forces** (the block below).
7	`st` then a blank line	`Enter filename, or <return> for screen output` : the blank line chooses the screen. Then **Stability-axis derivatives** (below).
8	a blank line, then `quit`	Back out of the menu, then AVL closes.

The screen after `x` (real output, trimmed)

```
Vortex Lattice Output -- Total Forces
 Sref = 0.17548E-01   Cref = 0.47406E-01   Bref = 0.40000
 Xref = 0.36970E-01   Yref =  0.0000       Zref =  0.0000
 Alpha =   4.39025     pb/2V =  -0.00000     p'b/2V =  -0.00000
 CXtot =   0.01937     Cltot =  -0.00000     Cl'tot =  -0.00000
 CYtot =  -0.00000     Cmtot =   0.00007
 CLtot =   0.30000
```

The screen after `st` (the neutral point is the last line)

```
Stability-axis derivatives...
                            alpha               beta
                    ----------------    ----------------
z' force CL |    CLa =   4.992724    CLb =  -0.000000
y  force CY |    CYa =   0.000000    CYb =  -0.000705
x  force CD |    CDa =   0.087892    CDb =   0.000000
x' mom.  Cl'|    Cla =  -0.000000    Clb =  -0.040653
y  mom.  Cm |    Cma =  -0.499535    Cmb =   0.000000
z' mom.  Cn'|    Cna =  -0.000000    Cnb =   0.003215
Neutral point  Xnp =   0.041713
```

Step 2 (continued): read three numbers

Number on screen	Value	Plain meaning
`Xnp`, neutral point	**0.041713 m = 41.7 mm**	Distance behind the root leading edge. The CG must sit *ahead* of this. (MODEL: AVL is inviscid; its neutral point can be off by a few percent of the chord.)
`CLtot`	**0.30000**	The lift coefficient you asked for with `a c 0.3`. Alpha came out as 4.39025 degrees.
`Cmtot`	**0.00007**	Pitching moment about the CG (37.0 mm, the file's reference point). Near zero means the 6 degree tail bend balances this CG at this lift. Positive: nose-up left over; negative: nose-down.
`Cref`, `Sref`	47.4 mm, 17,548 mm^2	Chord and wing area AVL uses (MAC and wing area of Swift A at 400 mm).
`Clb`, `Cnb`	-0.040653, 0.003215	Roll and yaw stiffness per radian. `Cnb` positive means it weathercocks; the project's rough floor is 0.004, and Swift A is below it (see the fin kit).

> **Do not use AVL's `MODE` menu.** Typing `mode` then `n` (calculate eigenmodes) makes AVL 3.52 crash on this machine. The same crash happens with the root-locus commands. The crash seen here:

```
 iter d(alpha)   d(beta)    d(pb/2V)   d(qc/2V)   d(rb/2V)
   1  0.637E-15 -0.000E+00  0.000E+00 -0.000E+00 -0.000E+00

 ** Zero Velocity.  Specify with run file or M menu

 Eigenmodes not computed for run case           1

Program received signal SIGSEGV: Segmentation fault - invalid memory reference.
```

Exit code of that AVL run: 3 (non-zero = crashed). The project gets spiral and Dutch-roll numbers by writing AVL's system matrix with the `S` command and solving it in Python (see `fin_kit_v4.py`).

Check that you can repeat it

Run `.\tools\avl\avl352.exe` again, retype the same lines, and change one thing: `a c 0.5` instead of `a c 0.3`. Real result here: alpha 6.69 degrees, CLtot 0.50000, `Xnp` 41.70 mm (against 41.71 mm at CL 0.3: the neutral point belongs to the shape and hardly moves), `Cmtot` -0.01992 (no longer zero: this CG and bend balance only near CL 0.3).

Not run by me: AVL's other commands (geometry plot, Trefftz plot) open a graphics window; they were switched off in step 2 on purpose.

Step 3: from the neutral point to the CG and the nose weight

This is arithmetic, done by `walk_balance.py` with the same functions the build uses. You type the neutral point you just read (**41.713** mm) and the margin you want (0.10 = 10% of the chord).

#	Do this	You should see
1	`py -B rigid_glider\walk_balance.py 41.713 0.10`	Five labelled steps (below). Nothing is written to disk.

```
STEP 1  static margin in millimetres
   MAC (the 'Cref' line on the AVL screen)      = 47.406 mm
   margin 0.10 x MAC                          = 4.741 mm
STEP 2  CG = neutral point - margin
   41.713 - 4.741                        = 36.972 mm behind the root leading edge
STEP 3  the glider before any nose weight (foam sheet + printed rail)
   foam sheet     6.651 g   (sheet area x foamboard areal mass, ASSUMED 300 g/m2)
   printed rail   1.911 g
   together M0    8.563 g, balancing at cg0 = 43.95 mm
STEP 4  how much weight at the nose hole to move the balance point back to the target
   target CG      36.97 mm;  nose weight position (rail hole) -12.5 mm
   weight = M0 x (target - cg0) / (position - target)
          = 8.563 x (36.97 - 43.95) / (-12.5 - 36.97)
          = 1.207 g
   all-up mass    9.770 g   (nose weight is 12.4 % of it)
   wing loading   5.46 N/m2
STEP 5  first-swoop depth estimate (formula, MODEL)
   depth = 3 x (W/S) / (rho x g x CL) = 3 x 5.46 / (1.225 x 9.80665 x 0.3) = 4.55 m
```

What happened	In words
Steps 1-2	The CG sits one tenth of a chord ahead of the neutral point: 41.713 − 4.741 = **36.97 mm**, which the build files round to **37.0 mm** (v4's `outlines.csv` lists 23.1 mm at 250 mm span; ×1.6 = 37.0).
Step 3	The foam sheet and the printed rail alone balance at 43.95 mm, *behind* the target. So weight must go forward.
Step 4	Weight needed at the nose hole = M0 × (target − cg0) / (hole − target). Put **1.21 g** at the nose hole, 12% of the 9.77 g all-up mass.
Step 5	A first guess at the drop: 3 × wing loading / (air density × g × CL). **4.55 m** if the lift coefficient stays at 0.3.

> Where the sheet's depth range **5 to 18 m** for Swift A foam 400 comes from: the formula gives the low end, 4.55 m. The high end multiplies it by the pull-out study's simulation-to-formula ratio for the light foam bird (2.05) and by 1.95 for a tail only half as effective as AVL says: 4.55 × 2.05 × 1.95 = 18.2 m. Swift A is **not** simulated by the pull-out program (it handles the flat kit wings and an older bird outline). So 5 to 18 m is a formula range, a MODEL estimate, not a prediction of a drop.

Foam sheet mass assumes 300 g/m² foamboard (ASSUMED, to be weighed). The printed rail and the nose hole position come from the v4 build. Your real glider will differ: weigh it and measure the balance point on test day (step 6).

Step 4: the pull-out simulation, one drop

The simulation drops a wing nose-down from rest and follows it for 30 seconds in still air (pitch only). It knows the three flat kit wings: `F45_span250`, `F60_span250`, `B45_span400`. The command takes three inputs: the wing, the CG (mm behind the root leading edge; a whole-mm kit setting) and the tab angle (degrees, trailing edge up).

#	Do this	You should see
1	`py -B rigid_glider\walk_pullout_one.py F45_span250 39 6`	Three lines of progress, then the result below. About 2 seconds.
2	`py -B rigid_glider\walk_pullout_one.py F45_span250 39 0` (same wing and CG, tab 0)	The same drop with no nose-up tab: it dives.
3	Open `results\runs\pullout_one\trajectory.svg` (double-click)	A side-view plot of the drop. `one_drop.csv` holds the headline numbers; `trajectory.csv` the path.

Tab 6 degrees

```
1. loading F45_span250 and running AVL for its derivatives (3 short AVL runs) ...
2. dropping it nose-down from rest: CG 39 mm, tab 6 deg ...

   glider mass            17.97 g
   static margin          +8.6 % of the chord (CG ahead of the neutral point = positive)
   VERDICT                SETTLES TO A GLIDE
   first-swoop depth      14.1 m   (lowest point reached before the first level-off)
   formula depth          12.0 m   (3 W/S / (rho g CL), a cross-check)
   settled flight path    -7.7 deg   glide ratio 7.4
```

Tab 0 degrees

```
1. loading F45_span250 and running AVL for its derivatives (3 short AVL runs) ...
2. dropping it nose-down from rest: CG 39 mm, tab 0 deg ...

   glider mass            17.97 g
   static margin          +8.6 % of the chord (CG ahead of the neutral point = positive)
   VERDICT                DIVES
   first-swoop depth      -   (lowest point reached before the first level-off)
   formula depth          -   (3 W/S / (rho g CL), a cross-check)
   settled flight path    -90.0 deg   glide ratio 0.0
```

Line	How to read it
VERDICT	`SETTLES TO A GLIDE` = it recovered. `DIVES` = it never pulled out. Other words: TUMBLES, DEEP STALL, NOT SETTLED.
first-swoop depth	Height lost before the first level-off: **14.1 m** for this kit wing. Compare with the formula line (a cross-check).
static margin	CG ahead of the neutral point, in % of the chord. Under 3% the model says it cannot tell.

What you change: only the command line. Second number = CG (try 41, then 36). Third = tab angle (try 2, 4, 8). First word = which wing. The same result as the project's full study: its `cases.csv` row for this wing, CG 39 and tab 6 reads 14.1 m, margin 8.59%, SETTLES TO A GLIDE.

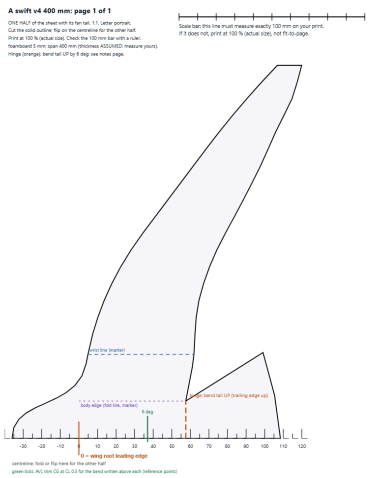
Optional, the whole study in about 18 seconds: `py -B rigid_glider\pullout_sim.py --out results\runs\pullout_practice --quick`, then open `results\runs\pullout_practice\index.html`. Leave off `--quick` for the full study; use `--out` every time so it writes to the practice folder.

Step 5: making the parts

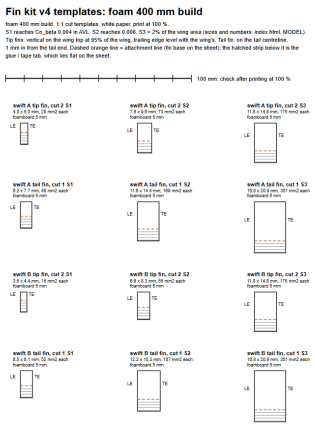
The PDFs to print are ready in `print_these\` (copied from the project). Printing is the normal route; regenerating is only needed if the outline changes.

Print and cut

#	Do this	You should see
1	In File Explorer open `print_these`. Double-click `A_templates.pdf`.	A 4-page PDF opens in your PDF viewer: foam 400 mm (page 1 outline, page 2 notes), then printed 250 mm (pages 3 and 4).
2	Press `Ctrl` + `P`. Choose your printer and paper **Letter**. Set the scale to **100% / Actual size**, **not** 'Fit to page'. Print pages 1 and 2. Then lay a ruler on the **100 mm scale bar** (top right of each sheet): it must measure exactly 100 mm; if not, print again.	The print dialog; the exact words vary by PDF viewer (not checked on your viewer).
3	Cut out the solid outline on page 1. It is **one half** with its fan tail. Flip it on the centreline to draw the other half on the foam.	A swept half wing with a fan tail on the right (picture).
4	Mark the hinge (orange dashed), the wrist line and the body edge (markers only: do not cut them). Bend the tail up with the 6 degree wedge on page 2.	The wedge on the notes page; its note reads 'Score the top face, bend up to the gauge, glue the crease.'



A_templates.pdf, page 1: one half of Swift A, foam 400 mm, with the 100 mm bar (real render).



fin_templates.pdf, page 1: fin shapes for foam 400 mm. Orange dashed line = attachment line; hatched strip = glue or tape tab.

Fins (only if it wanders in yaw)

Print `print_these\fin_templates.pdf` the same way (100%, check the bar). Page 1 is foam 400 mm, page 2 printed 250 mm. Each box names the fin: for example *swift A tip fin, cut 2, S3*. The sheet `drop_test_sheet.pdf` says to try **tip fins, S3, foam 400** first.

Optional: regenerate the files (inside the practice folder)

`py -B rigid_glider\bird_outline_v4_swift.py --out results\runs\v4_practice` took 3 minutes here (52 serial AVL runs) and wrote A_templates.pdf, B_templates.pdf plus the cut files and tables. `py -B rigid_glider\fin_kit_v4.py` took 2 minutes and writes `results\runs\rigid_glider\fin_kit_v4\fin_templates.pdf` inside the practice copy.

> **Do not add** `--quick` **to the v4 script:** it writes the template PDFs, then stops with an error (`TypeError: 'NoneType' object is not subscriptable`). The full run finishes cleanly.

Step 6: mock test day, the sheet

Print `print_these\drop_test_sheet.pdf` (2 pages, B5, one-sided). Page 1 is the plan; page 2 is the log. Before the first drop do the bench check on page 1 (tick the boxes): mass, CG, bend, warp, symmetry, hook.

> **EXAMPLE** Everything in the row below is **made up** to show how to fill it in. It is not a result.

#	Article	CG mm	Bend deg	Height m	Outcome	Swoop depth m	Glide dist m	Clip	Note
1	swift A foam 400	37.0	6	10	D	-	-	IMG_0001	dived, no recovery
2	swift A foam 400	39.4	6	10	G	6.5	14	IMG_0002	clean glide, slight turn right

EXAMPLE rows. Outcode codes: **G** pulls out and glides, **PS** pulls out then porpoises or stalls, **D** dives, **T** tumbles, **SP** spirals or turns away, **RF** release fault, **H** hand glide. The starting CG 37.0 mm, bend 6 degrees and depth range 5 to 18 m for Swift A foam 400 are the real settings from page 1 of the sheet (step 3).

What to do between drops (the trim guide on page 1 of the sheet)

What happened	Adjustment
Dives / no recovery	CG aft one step, or bend up one gauge step
Porpoise / stall / growing oscillation	CG forward one step, or bend down one gauge step
Turns one way	Fix warp first, then a small trailing-edge trim
Wanders or spirals	Tip fins or a small fin
Swoop deeper than predicted	Check bend on the gauge and CG on the balance

A step is 2 to 3 mm of CG (JUDGEMENT, from the sheet) or the next angle on the field-pack gauge (2, 2.5, 4, 5, 6, 7.5, 8 degrees). **Change one thing between drops.** Example from the dive in row 1: move the CG one step aft (about 2.4 mm: 37.0 to 39.4 mm) and drop again, as in row 2.

Re-running the arithmetic for a new CG. A smaller margin puts the CG further back. `py -B rigid_glider\walk_balance.py 41.713 0.08` (8% margin, the back end of the sheet's model range) gives:

```
STEP 2  CG = neutral point - margin
   41.713 - 3.793                        = 37.920 mm behind the root leading edge
   target CG      37.92 mm;  nose weight position (rail hole) -12.5 mm
   weight = M0 x (target - cg0) / (position - target)
          = 8.563 x (37.92 - 43.95) / (-12.5 - 37.92)
          = 1.024 g
   all-up mass    9.586 g   (nose weight is 10.7 % of it)
```

The model range is 8% to 15% margin (CG 34.6 to 37.9 mm). Stepping beyond the back end (past 37.9 mm) goes outside the range the model is used for: do it only while it still pulls out, as the sheet says. Then **measure the real CG on the balance** and write it in the CG column; do not copy the plan.

Step 6 (continued): the drop tracker



The tool in its own demo mode (a made-up path): the numbers shown are demo values, not a drop. Real screenshot.

It is one web page that works offline. Your video never leaves the browser.

Open it: in File Explorer double-click `tools\drop_tracker\drop_tracker.html` in the practice folder.

Self-test (add `?selftest` to the address): **SELFTEST PASS 34/34**.

#	Click this
1	**Choose File** (Video box), pick the phone video. Set **Frame rate** to 30 or 60 (buttons).
2	**Set scale**. Click both ends of the known reference (a marked pole), then type its length in metres in **Reference length (m)**.
3	Optional: **Set level line**, click two points on the horizon or ground.
4	**Track**. Step to the release frame (`,` `.` or **< Frame** / **Frame >**). Click the glider. The video jumps N frames; click it again, every step, to the end.
5	Click a point in the list, then tag it: **Release**, **Lowest**, **Level-off**, **Glide start**, **Glide end**. (**Lowest: auto** picks the deepest.)
6	Read **Results**: swoop depth, glide distance, glide ratio and angle.
7	Fill **Drop #**, **Article**, **CG (mm)**, **Bend (deg)**, **Drop height (m)**, **Outcome** (G, PS, D, T, SP, RF, H), **Note**. Click **Download summary CSV** (and **Download points CSV**).

What you get: two files named after the video, `<video>_summary.csv` and `<video>_points.csv`. The summary is one row whose columns match the log on the sheet:

```
drop #,article,CG,bend,height,outcome,swoop depth,glide dist,clip,note
```

EXAMPLE row: `2,swift A foam 400,39.4,6,10,G,6.5,14,IMG_0002,clean glide`. **Copy summary row** puts it on the clipboard to paste into a spreadsheet.

Limits: the scale holds only in the plane of the reference; keep the camera side-on and the reference at the flight path's distance. Heading cannot be read from one camera.

Not run by me: the clicks above (a person has to click a video). They are copied from the tool's own **Steps** panel and its button names; the self-test and demo page were run with headless Edge.

Step 7: CFD (half a page)

What it is for. Checking AVL against the wind-tunnel plate (a flat rectangular plate, Re 60,000) to see whether AVL's pitching moment and centre of pressure can be trusted. It is **not** run on our wings, and it is not used for balance or pull-out. The runs live in the WSL distro `MMAV-CFD`; do not start or stop one while another is running.

Status, stop and start. Copied exactly from `cfd\validation_sweep\refined_8deg\README.md` (PowerShell; **I did not run them**):

```
Status: wsl -d MMAV-CFD -u cfd -- bash /home/cfd/run/ref/ref_status_a08.sh
Stop:   wsl -d MMAV-CFD -u cfd -- bash /home/cfd/run/ref/ref_stop_a08.sh
Start:  Start-Process -WindowStyle Hidden wsl.exe -ArgumentList '-d','MMAV-CFD','-u','cfd','--
','bash','/home/cfd/run/ref/ref_launch_a08.sh'
```

The README adds: stop affects only that run; relaunching with the start command resumes from the latest complete time; a hidden Windows `wsl.exe` keeps the distro alive, so killing it, sleeping, restarting or logging off stops the run.

Open a case in ParaView (installed at `C:\Program Files\ParaView 5.12.0`; the path exists, I did not start it):

#	Do this	You should see
1	Start menu, type `ParaView`, Enter.	The ParaView window.
2	**File** > **Open**. In the file-name box type `\\wsl.localhost\MMAV-CFD\home\cfd\run\vp_ref_a06\case.foam` (swap in another case name) and press Enter.	The case appears in the Pipeline Browser. (Case name from the README; the `case.foam` file is not checked here.)
3	Click the green **Apply** button.	The plate's mesh appears; pick the field (U, p) in the toolbar.

The ParaView clicks are standard ParaView menus, not run by me. The 6 degree case is finished; the 8 degree case is the one being run or stopped.

Step 8: cheat sheet, every command

```
PowerShell, once:
  cd "C:\Users\Kai\OneDrive - purdue.edu\Purdue\AOL\MAV-2026\Codes"
  py -B rigid_glider\make_practice_folder.py C:\Users\Kai\Documents\mmav_practice
  cd C:\Users\Kai\Documents\mmav_practice          (stay here)

AVL (type each line + Enter):
  .\tools\avl\avl352.exe
  plop / g / (blank)
  load practice_inputs\SwiftA_foam400_bend6.avl
  oper / a c 0.3 / x / st / (blank) / (blank) / quit      (never: mode, n)

Balance:    py -B rigid_glider\walk_balance.py 41.713 0.10
Pull-out:   py -B rigid_glider\walk_pullout_one.py F45_span250 39 6      (wing, CG mm, tab deg)
            py -B rigid_glider\pullout_sim.py --out results\runs\pullout_practice --quick
Regenerate: py -B rigid_glider\bird_outline_v4_swift.py --out results\runs\v4_practice      (no --quick)
            py -B rigid_glider\fin_kit_v4.py
Print:      print_these\A_templates.pdf, fin_templates.pdf, drop_test_sheet.pdf   (100%, check the bar)
Tracker:    double-click tools\drop_tracker\drop_tracker.html
CFD:        status / stop / start commands on this page (do not run unless you mean to)
```